PIMPRI-CHINCHWAD EDUCATION TRUST'S

Pimpri-Chinchwad College of Engineering

(An Autonomous Institute) • Affiliated to Savitribai Phule Pune University (SPPU) • ISO 21001:2018 Certified by TUV

ASSIGNMENT NO 4

Name:	**Siddhesh Parkar**	**Course:**	Design and Analysis of Algorithm
PRN:	**124B1F021**	**Class:**	TY IT A
Topic:	**Dijkstra's Algorithm (Emergency Vehicle Routing)**	**Batch:**	A1

1. Problem Statement

Smart Traffic Management for Emergency Vehicles: In a modern smart city infrastructure, efficient emergency response is critical. An ambulance is stationed at Source intersection *S* and must reach the nearest hospital *D* in the shortest possible time. Road segments experience variable traffic congestion, dynamically modifying travel times in real time. Implement **Dijkstra's Algorithm** using a priority queue to compute shortest paths and dynamically recalculate optimal navigation routes upon traffic updates.

2. Course Objectives

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcomes

- Analyze the asymptotic performance of algorithms.
- Solve computational problems by applying suitable paradigms of Divide and Conquer or Greedy methodologies.

4. Theory

Dijkstra's algorithm is a single-source shortest path algorithm for directed or undirected graphs with non-negative edge weights. It iteratively expands the frontier of known shortest distances from the source node using a greedy relaxation strategy.

Core Principles:

- **Greedy-Choice Property:** At each step, the algorithm chooses the unvisited vertex *u* with the minimum tentative distance. Because all edge weights are non-negative, this distance is guaranteed to be minimal and immutable once extracted from the priority queue.
- **Edge Relaxation:** For each neighbor *v* of *u* connected with weight *w*, if `dist[u] + w < dist[v]`, the distance to *v* is updated to `dist[u] + w` and `parent[v]` is set to *u*.

Complexity Analysis:

- **Priority Queue (Min-Heap):** Extract-min takes O(log V). Each of the *V* vertices is extracted once (O(V log V)).
- **Edge Relaxation:** Each of the *E* edges is relaxed at most once, taking O(log V) per priority queue push.

- **Total Time Complexity: O((V + E) log V)**.
- **Space Complexity: O(V + E)** using an adjacency list and distance/parent arrays of size *V*.

5. Algorithm & Pseudo Code

```
procedure dijkstra(source, graph, dist, parent):
    dist.assign(V, ∞)
    dist[source] ← 0
    Create priority_queue pq storing pairs (distance, vertex)
    pq.push((0, source))
    while pq is not empty do
        (d, u) ← pq.top(); pq.pop()
        if d > dist[u] then continue
        for each neighbor (v, w) in graph[u] do
            if dist[u] + w < dist[v] then
                dist[v] ← dist[u] + w
                parent[v] ← u
                pq.push((dist[v], v))
            end if
        end for
    end while
end procedure
```

6. Program Source Code (C++)

```
/*
 * Student Name: Siddhesh Parkar
 * PRN: 124B1F021
 * GitHub Repository: https://github.com/siddhesh160606/DAA_lab
 * Assignment 4: Ambulance Route Optimization to Nearest Hospital using Dijkstra's Algorithm
 */

#include <iostream>
#include <vector>
#include <queue>
#include <climits>
using namespace std;

void dijkstra(int source, vector<vector<pair<int,int>>> &graph, vector<int> &dist) {
    int V = graph.size();
    dist.assign(V, INT_MAX);
    dist[source] = 0;

    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0, source});

    while (!pq.empty()) {
        int u = pq.top().second;
        int d = pq.top().first;
        pq.pop();

        if (d > dist[u]) continue;

        for (auto &edge : graph[u]) {
            int v = edge.first;
            int w = edge.second;

            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}

int main() {
    cout << "Student Name : Siddhesh Parkar\n";
    cout << "PRN          : 124B1F021\n";
    cout << "GitHub Link  : https://github.com/siddhesh160606/DAA_lab\n";
    cout << "------------------------------------------------------\n";

    int V, E;
    cout << "Enter number of intersections (vertices): ";
    cin >> V;

    cout << "Enter number of roads (edges): ";
    cin >> E;

    vector<vector<pair<int,int>>> graph;
    graph.resize(V); // Allocate memory for V vertices

    cout << "Enter edges (u v w):\n";
    for (int i = 0; i < E; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        graph[u].push_back({v, w});
        graph[v].push_back({u, w}); // undirected road
    }

    int source;
```

```
    cout << "Enter ambulance start location (source): ";
    cin >> source;

    int H;
    cout << "Enter number of hospitals: ";
    cin >> H;

    vector<int> hospitals(H);
    cout << "Enter hospital nodes: ";
    for (int i = 0; i < H; i++) {
        cin >> hospitals[i];
    }

    vector<int> dist;
    dijkstra(source, graph, dist);

    int minTime = INT_MAX, nearestHospital = -1;

    for (int h : hospitals) {
        if (dist[h] < minTime) {
            minTime = dist[h];
            nearestHospital = h;
        }
    }

    if (nearestHospital == -1)
        cout << "No hospital reachable.\n";
    else
        cout << "Nearest hospital is at node " << nearestHospital
             << " with travel time " << minTime << " minutes.\n";

    return 0;
}
```

7. Sample Execution Output

```
============================================================
Student Name: Siddhesh Parkar
PRN         : 124B1F021
GitHub Repo : https://github.com/siddhesh160606/DAA_lab
Assignment  : 4 - Ambulance Route Optimization using Dijkstra's Algorithm
============================================================

Enter number of intersections (vertices): 6
Enter number of roads (edges): 7
Enter edges (u v w):
0 1 4
0 2 2
1 2 1
1 3 5
2 3 8
2 4 10
3 5 6

Enter ambulance start location (source): 0
Enter number of hospitals: 2
Enter hospital nodes: 3 5

Nearest hospital is at node 3 with travel time 3 minutes.
```

8. Conclusion

Dijkstra's shortest path algorithm combined with a binary heap priority queue provides an optimal, fast (O((V + E) log V)) routing mechanism for urban emergency vehicles. By tracking predecessor nodes in a parent array, the system reconstructs exact turn-by-turn navigation paths and dynamically adapts to real-time traffic jams by re-evaluating optimal hospital targets within milliseconds.

GitHub Repository Link

> **Repository Link:** https://github.com/siddhesh160606/DAA_lab